

Classifying water scenes in satellite images

Pranav Tripathi · thesis assignment · EuroSAT dataset

Introduction

I started from a straightforward question: if I am given a small satellite photograph, can I tell whether the scene is mainly water or not? I used a public set of Sentinel-2 tiles. Each tile is 64 by 64 pixels and has thirteen spectral bands. There are only one hundred labelled tiles in total. Twenty of them are water scenes (rivers or lakes). The rest are other land-cover types such as crops, forest, roads or buildings.

It is important to be clear about the label. I am doing scene classification, not segmentation. Each tile gets one answer: water scene or not. That answer follows the original land-cover class of the whole tile. So a highway photograph that happens to show a river is still a non-water example for me. The scene is a road. If my model calls it water because it saw the river, that is a mistake under these labels, even though a mapping product might want the opposite. I am not drawing a shoreline or measuring how much of the tile is wet.

Table 1 is the split I used. Sixty tiles for training, twenty to make decisions, twenty held out for a final look. That leaves only four water tiles in the test set. From the beginning I treated that as a warning: one lucky score on twenty images would not mean the problem was solved.

Split	Tiles	Water	Not water
Training	60	12	48
Validation	20	4	16
Test	20	4	16
All	100	20	80

Table 1. How the hundred tiles are divided. The full counts are in `results/split_summary.json` on GitHub.

How I approached it

I did not begin with a large neural network. Eighty of the hundred tiles are not water, so a method that always says “not water” is already right most of the time. I wanted something that actually finds water, and I wanted each step to be justified before I added complexity. I therefore built three models, one after the other, on the same hundred tiles. They are compared, not combined. Nothing is stacked or voted at the end.

The first idea came from the physics of the sensor. Water looks bright-ish in green light and very dark in the near-infrared and short-wave infrared. Remote sensing has used that fact for a long time, through indices such as NDWI, which compares the green band to the near-infrared band, and MNDWI, which compares green to short-wave infrared. For each tile I summarised those index maps with a few numbers: the mean, the maximum, and how much of the tile looks wet. A small logistic regression then learns a line in that space. I also tried a single cut on the mean index, chosen only on the validation tiles. I treated this spectral model as the reference. If a neural network could not beat it, there was no reason to prefer the network.

The second step was about shape. A thin river and a lake can have a similar average index but look different on the ground. So I trained a small convolutional network from scratch: a few layers, dropout, and early stopping on the validation loss. I kept it small on purpose. I was not trying to prove that the size is perfect for sixty examples, only that a network with far fewer weights than a ResNet can be tested fairly here. I did not add extra images or flips. The constraint was to stay with the original hundred tiles.

The third step used a ResNet that had already been trained on unlabelled Sentinel-2 imagery. I froze that backbone and trained only a linear head on top of the embeddings. The large network is then a feature extractor, not something I am fitting on sixty labels. I showed it different band sets — visible colour, the water-related bands, or all thirteen — by hiding unused channels rather than rebuilding the network each time. The question here is whether a pretrained satellite representation adds anything once the spectral model is already on the table.

Figure 1 is why the first step felt natural. In true colour a lake can look like a dark grey surface. In the infrared composite the same water goes almost black, while plants stay bright. That is the signal the spectral model is built on.

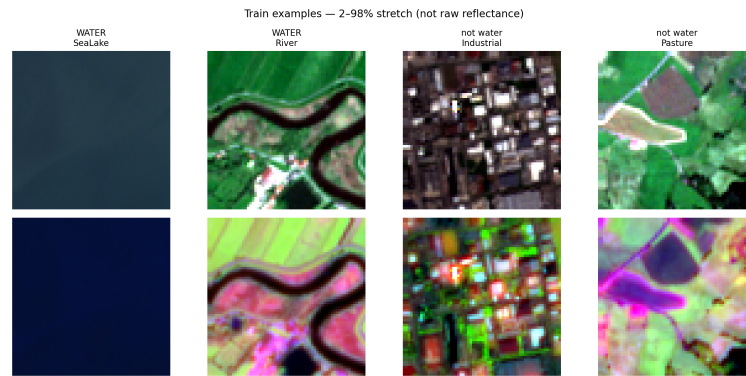


Figure 1. Four training tiles. Top row is true colour. Bottom row is a water composite using short-wave infrared, near-infrared and green. The lake and the river go dark in the lower row; vegetation does not. Image file: results/fig_examples.png.

How I decided to score the models

Because most tiles are not water, I refused to rank methods by accuracy alone. Accuracy is easy to explain, and it is useful as a check: anything below the “always not water” answer is worse than doing nothing. But it can look strong while the model never finds a river. Precision tells me whether a water call is trustworthy, yet a timid model can look precise by almost never calling water. Recall tells me how many real water scenes I catch, but with only four test positives a single miss moves that number a lot. F1 sits between precision and recall and is the thresholded number I quote most often. I also kept a ranking score that does not depend on the cut, because choosing the cut on four validation water tiles is itself unstable. The ranking score can be perfect while the cut still misses one scene. I see that as a result, not as a nuisance.

Cuts and other choices were made on the validation tiles only. The test tiles were used once. I also repeated the training in five folds on the same hundred tiles, not to pretend I had more data, but to see whether a good-looking test score survived a different split. Pairwise tests between models on twenty images are in the repository; they do not pick a winner, which is what I expected with four positives.

Model	Accuracy	F1	Ranking score
Always predict not water	0.80	0.00	0.20
Simple water-index cut	0.95	0.86	0.89
Logistic on water-index features	0.95	0.86	1.00
Small CNN	1.00	1.00	1.00
Frozen ResNet, colour only	0.80	0.33	0.65
Frozen ResNet, all bands	1.00	1.00	1.00

Table 2. Official twenty-tile test set. The ranking score is average precision. The full table with intervals and extra band sets is results/official_metrics.md on GitHub.

What I found

I read the results as a chain, not as a leaderboard. The first number that matters is **0.80**. That is the accuracy of always saying “not water”. Table 2 starts there so that every later score has to mean more than “most tiles are land”.

The second number is **0.86**. That is the F1 of the spectral logistic model, and also of a plain cut on the water index. So before any neural network, the physics I started with already finds water without drowning in false alarms. The ranking score for that logistic model is perfect on these twenty tiles: it puts the four water scenes at the top of the list. The F1 is not perfect because the cut chosen on validation still misses one of them. That is how I learned not to treat a single threshold as the whole story.

The third number looks better at first: the small CNN can reach **1.00** on this particular test split. If I had stopped there I would have said the network won. I did not stop there. When I reused the same hundred tiles in five folds, that CNN dropped to about **0.63** on average and jumped around from fold to fold. The perfect test score was a small split, not a stable method. Figure 2 is that check. The spectral model and the frozen ResNet stay near each other. The small CNN is the one with the long error bar.

The fourth number is the one I trust for the pretrained model: around **0.8** in cross-validation, in the same range as the spectral logistic model. Colour-only features are clearly weaker in Table 2, which matches Figure 1: visible light is not where water stands out. Adding the infrared bands brings the frozen network back in line with physics. It does not clearly overtake it.

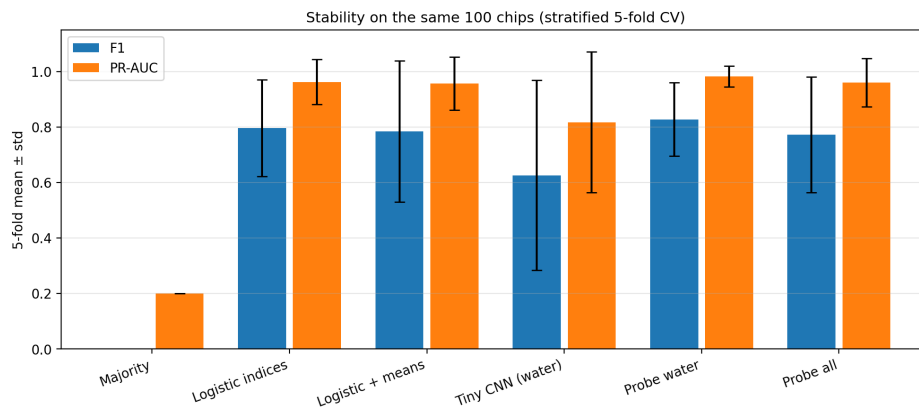


Figure 2. The same hundred tiles, split five ways. The small CNN moves around much more than the spectral model or the frozen ResNet. Image file: [results/fig_cv_stability.png](#).

Model	Average F1 across folds
Always predict not water	0.00
Logistic on water-index features	0.80, with a spread of 0.17
Small CNN	0.63, with a spread of 0.34
Frozen ResNet, water-related bands	0.83, with a spread of 0.13
Frozen ResNet, all bands	0.77, with a spread of 0.21

Table 3. Cross-validation on the original hundred tiles. Full fold values are in [results/cv_f1.json](#) on GitHub.

Closing

I approached the problem as a sequence of decisions. First use what is already known about water in Sentinel-2. Then ask whether a small network can add shape. Then ask whether a frozen satellite ResNet adds a representation I could not learn from sixty labels. I judged them with F1 and a ranking score because accuracy is too easy when eighty tiles are land. The chain of results is: the trivial answer is already 0.80 accurate; physics already reaches an F1 of 0.86; a 1.00 on twenty tiles did not survive resampling; a frozen ResNet lands in the same place as physics. With this many labels I would keep the spectral model as the reference. If more labelled tiles arrived later, I would keep the same order and only then unfreeze the large network.

The other plots and tables from the experiments — band comparisons, precision–recall curves, confusion matrices, training curves, and the tiles each model got wrong — are not repeated here. They are in the results folder of the GitHub repository https://github.com/pranavtripathi6844/thesis_assignment-.